

Observability for Robots

An Evaluation of OpenTelemetry for Real-Time and Safety-Critical Robotic Systems

Report No. XM-TR-2026-01
Version 1.0
Status Draft for review
Date July 2, 2026
Prepared by xMotion Engineering

EXECUTIVE SUMMARY

Cloud-native observability has converged on OpenTelemetry (OTel) as a vendor-neutral standard for traces, metrics, and logs. Robotics teams, seeking better tools for post-incident reconstruction and issue localization, are tempted to adopt it wholesale. This report evaluates how much of OpenTelemetry actually transfers to a robotic system, from the joint perspective of a cloud-native and a hard-real-time/safety-critical engineer. We argue that the recurring confusion stems from treating “observability” as a single concern. We decompose a robot’s telemetry needs into **three planes** — a **safety plane**, a **recording plane**, and an **observability plane** — and show that OpenTelemetry cleanly owns only the last. We provide a component-by-component evaluation, a quantitative analysis of the timescale, loss-tolerance, and causality mismatches between cloud and robotic workloads, and five case studies drawn from published measurements (Dapper, ros2_tracing/LTTng, PREEMPT_RT, ROS 2/DDS communications, and Pressure Stall Information). We conclude that OpenTelemetry is a strong **adopt** for the observability plane and its transport ($\approx 40\%$ of the problem), a **borrow-and-adapt** for dataflow tracing and health metrics ($\approx 30\%$), and irrelevant-to-harmful for the real-time hot path and the safety monitor ($\approx 30\%$). We enumerate the gaps a robotics stack must fill — real-time-safe capture, a hardware time model, lossless anomaly recording, and dataflow-native causality. A prior-art survey then shows these gaps are met by mature, framework-agnostic building blocks (LTTng, NanoLog/Quill, MCAP, OpenTelemetry), so the design task is not to build an engine but to bind them coherently. We propose a lightweight strategy — one thin, OTel-shaped instrumentation surface plus a common spine, with the engines as pluggable, compile-time-selected sinks — and map it onto the xMotion component family.

Keywords — observability · OpenTelemetry · real-time systems · robotics · safety-critical · telemetry · flight recorder

Contents

1	Introduction	2
1.1	Contributions	2
1.2	Scope and non-goals	2
2	Background: What OpenTelemetry Is	2
3	Background: Real-Time and Safety-Critical Constraints	3
4	An Analytical Framework: The Three-Plane Model	3
5	Component-by-Component Evaluation	4
6	Quantitative Analysis of the Mismatch	5
6.1	Timescale: four to six orders of magnitude	5
6.2	Cardinality and aggregation destroy forensic value	6
6.3	Overhead and determinism	6
7	Case Studies	6
7.1	Dapper: sampling is a feature of scale, not of correctness (cloud)	6
7.2	ros2_tracing / LTTng: real-time tracing is possible, but it is not OTel (robotics)	7
7.3	PREEMPT_RT and OSADL latency plots: the hiccup budget is microseconds (real-time)	7
7.4	ROS 2 / DDS communications: dataflow latency is real and jitter-sensitive (robotics) ..	7
7.5	Pressure Stall Information: the cloud-native community built the right hiccup detector (transferable)	7
8	Gap Analysis: What Robotics Must Add	7
9	Related Work: Existing Building Blocks	8
10	A Coherent, Lightweight Instrumentation Strategy	8
10.1	One surface, many sinks	9
10.2	What to own versus what to adopt	9
10.3	Three rules that keep it lightweight	9
10.4	Two data classes, one surface	9
10.5	ROS-free core, ROS-bridgeable edge	10
11	Recommended Architecture for the xMotion Family	10
12	Threats to Validity	11
13	Conclusion	11
A	Host telemetry data sources (Linux)	12
B	Proposed robot.* semantic-convention seeds	12
	References	12

1 Introduction

Robotic systems deployed in the field fail in ways that are hard to reproduce on the bench: a control loop that misses a deadline once every few hours, a sensor that goes stale for 40 ms during a thermal transient, a planner starved of CPU while a log rotation flushes to a slow SD card. Diagnosing these requires **observability** — the ability to reconstruct what the system was doing, across software and hardware, at the moment of failure. The cloud-native community has invested a decade building exactly this capability and has standardized it around OpenTelemetry (OTel), now one of the most active projects in the Cloud Native Computing Foundation [1].

It is therefore natural for a robotics team to ask: *should we adopt OpenTelemetry?* The answer that circulates informally — “yes, it does structured logs, metrics, and traces” — is incomplete and, applied naively, dangerous. OpenTelemetry was designed against a set of assumptions (millisecond request latencies, abundant and droppable traffic, remote-procedure-call causality, always-on connectivity) that hold in a data center and fail on a robot. Conversely, a reflexive “no, robots are different” discards a genuine gift from the cloud-native community: a mature, vendor-neutral transport and a rich backend ecosystem that robotics has never had.

1.1 Contributions

This report makes the disagreement precise. Specifically, we:

1. Decompose robot telemetry into a **three-plane model** (§4) that predicts which OpenTelemetry components fit and which do not.
2. Provide a **component-by-component evaluation** with an adopt/adapt/skip verdict and rationale for each of OpenTelemetry’s parts (§5).
3. Quantify the **timescale, loss-tolerance, and causality mismatches** between cloud and robotic workloads using published rates and measurements (§6).
4. Ground the argument in **five case studies** from the literature (§7).
5. Enumerate the **gaps** a robotics stack must fill beyond OpenTelemetry (§8) and map a concrete architecture onto the xMotion component family (§9).

1.2 Scope and non-goals

We consider host-side telemetry for a mobile robot: a multi-core Linux compute node (possibly with a GPU), one or more real-time control loops, a set of device drivers, and intermittent connectivity to a fleet backend. We treat the microcontroller/firmware tier only where it bears on safety independence. We do not evaluate specific commercial backends, and we do not benchmark the xMotion stack itself — our quantitative claims are drawn from published, reproducible measurements and are used to establish **order-of-magnitude** arguments, not point estimates (see §10, Threats to Validity).

2 Background: What OpenTelemetry Is

“OpenTelemetry” names three separable things, and conflating them is the source of most confusion.

Specifications a data model for three **signals** — **traces** (causally linked spans), **metrics** (counters, gauges, histograms with attributes), and **logs** (structured records) — plus **semantic conventions** that fix attribute names (e.g. `system.cpu.utilization`, `system.memory.usage`) [2], and a **context propagation** mechanism (W3C Trace Context [3]) for carrying trace identity across service boundaries.

Wire protocol the OpenTelemetry Protocol (OTLP) [4], a push-based, Protobuf-over-gRPC-or-HTTP encoding, with a high-throughput Arrow variant.

Implementation language SDKs split into a lightweight **API** (no-op unless an SDK is installed) and a heavier **SDK** (aggregation, batching, exporters); the vendor-neutral **Collector** (a

standalone process with receivers, processors, and exporters); and the backend ecosystem (Prometheus-compatible stores, Tempo/Jaeger, Grafana).

Three architectural properties matter for our evaluation. First, the **API/SDK split** means a library can instrument against a cheap API and let the application choose the heavy SDK and exporter. Second, the **Collector** can run as a sidecar that buffers and forwards, decoupling the workload from backend availability. Third, the model is fundamentally **sampling-oriented**: it assumes data volume so high that keeping all of it is neither necessary nor affordable — an assumption inherited directly from its intellectual ancestor, Google’s Dapper [5].

3 Background: Real-Time and Safety-Critical Constraints

Robotic control operates under constraints that observability tooling must respect rather than violate.

Determinism a torque or current loop runs at 1–10 kHz, i.e. a 100 μ s–1 ms period, with a jitter budget in the low microseconds. On mainline Linux, scheduling latency can spike into the milliseconds; PREEMPT_RT reduces worst-case latency to the tens of microseconds on tuned hardware [6], [7]. Any instrumentation on this path must be lock-free, allocation-free, and bounded — properties the OTel SDK does not provide.

Safety independence functional-safety standards (ISO 26262 [8], IEC 61508 [9], DO-178C [10]) require that a monitor detecting a hazardous condition be **independent** of the function it monitors, often on a separate execution partition (ARINC 653 [11]) or a separate processor with a hardware watchdog (AUTOSAR Watchdog Manager [12]). A diagnostic pipeline that can itself fail, block, or consume unbounded resources cannot be load-bearing for safety.

Dataflow causality a robot’s “unit of work” is not a request but a **pipeline** — sensor $\rightarrow$ perception $\rightarrow$ planning $\rightarrow$ control — often realized over shared memory or a publish/subscribe middleware such as DDS [13], pipelined and many-to-many, with no clean request boundary to anchor a trace.

Connectivity robots are mobile and frequently disconnected. A pull-based, always-reachable monitoring model does not apply; telemetry must be captured locally and shipped opportunistically.

4 An Analytical Framework: The Three-Plane Model

The central claim of this report is that “robot observability” is not one concern but three, distinguished by their loss tolerance, latency budget, and consumer (Table 1).

Plane	Loss tolerance	Latency / rate	Primary consumer
Safety	zero — must never miss a hazard	hard real-time (μ s)	the robot itself (failsafe)
Recording	zero around incidents — lossless	high-rate raw (10 Hz–10 kHz)	engineer, post-incident
Observability	tolerant — sampling/aggregation OK	seconds, aggregated	fleet operator, dashboards

Table 1: The three telemetry planes of a robotic system. OpenTelemetry is a native fit only for the observability plane.

Safety plane an independent health/watchdog channel that can trip a failsafe. It must be deterministic and verifiable and must not share the fate of what it monitors. **OpenTelemetry has no place here.**

Recording plane a lossless, high-rate, on-robot capture of raw signals and events for incident reconstruction — the software analogue of an aircraft flight recorder. Its defining requirement (keep everything around the rare event) is the **opposite** of sampling. **OpenTelemetry is the wrong tool; the robotics-native format is MCAP/rosbag2 [14].**

Observability plane aggregated, non-real-time, fleet-wide health, metrics, and cross-component correlation. Loss-tolerant, dashboard-facing. **OpenTelemetry owns this plane.**

The model’s predictive power is that every subsequent verdict follows from asking “which plane?” A component that assumes loss tolerance is fine on the observability plane and disqualifying on the safety plane; a format optimized for aggregation is right for observability and destructive for recording.

5 Component-by-Component Evaluation

Table 2 gives the verdict for each OpenTelemetry component; the rationale is developed below and quantified in §6–7.

Component	Verdict	Rationale
OTLP wire protocol	Adopt	Vendor-neutral and push -based; better than pull scraping for mobile/intermittent robots [4].
Collector (sidecar)	Adopt	On-robot store-and-forward, batching, routing; runs as a separate process (fault isolation).
Metrics data model	Adopt*	Ideal for aggregated health/SLO ; *not for high-rate raw signals (§6.2).
Semantic conventions (<code>system.*</code> , <code>hw.*</code>)	Adopt	Free, portable dashboards for host/OS/accelerator telemetry [2].
Resource model	Adopt	Fleet identity (robot id, version) for cross-robot queries.
Backend ecosystem	Adopt	Off-robot analysis; no reason to rebuild.
Traces / spans	Adapt	Keep causal correlation; export sampled/anomaly-triggered, never per control cycle (§6, §7.1).
Context propagation	Adapt	Keep trace-id correlation; change the carrier from HTTP headers to messages (§8).
Logs signal	Bridge	Least mature OTel signal; forward existing structured logs rather than migrate the hot path.
SDK on the hot path	Skip	Not lock-free/allocation-free/bounded (§3, §6.3).
Head-based sampling	Skip	Inverts the robotics requirement to keep the rare event (§7.1).
Pull scraping as transport	Skip	Assumes reachability; robots are mobile/air-gapped.
RPC auto-instrumentation, service mesh	Skip	No on-robot analogue.

Table 2: Adopt / adapt / skip verdicts for OpenTelemetry components in a robotics context.

6 Quantitative Analysis of the Mismatch

6.1 Timescale: four to six orders of magnitude

The single most important number is the ratio between how fast a robot’s inner loop runs and how slowly cloud observability samples. Table 3 collects representative rates.

Activity	Period / interval
Current/torque control loop	0.1–1 ms (1–10 kHz)
IMU sample	1–10 ms (100–1000 Hz)
LiDAR scan	50–100 ms (10–20 Hz)
Camera frame	16–33 ms (30–60 Hz)
Typical cloud request (Dapper domain [5])	1–1000 ms
Prometheus default scrape interval [15]	15 s
OTel metric export interval (SDK default)	60 s

Table 3: Representative operating rates. The robotic control path runs 10^4 – $10^6 \times$ faster than the default OpenTelemetry metric export interval.

A metrics pipeline sampling at $\frac{1}{60}$ Hz cannot observe a 1 kHz control loop; it can only observe **aggregates** of it. This is not a tuning problem — pushing the scrape interval to 1 kHz would multiply cardinality by the time series count and overwhelm any Prometheus-class store [15]. The correct inference is architectural: **aggregated health** and **high-rate raw signal** are different data products with different pipelines (§6.2).

6.2 Cardinality and aggregation destroy forensic value

OpenTelemetry metrics inherit Prometheus’s cardinality discipline: each unique combination of attribute values is a distinct time series, and best practice is to keep cardinality bounded [15]. Robotics telemetry is natively high-cardinality (per-joint, per-sensor, per-cycle) and high-rate. Forcing it through the metrics model requires aggregation (histograms, rates), which is **lossy by construction**: a p99 latency histogram cannot tell you **which** cycle overran or **what the IMU read** at that instant — precisely the facts needed for reconstruction. Hence the recording plane must preserve raw, timestamped samples (MCAP [14]), and the metrics plane must carry only their aggregates.

6.3 Overhead and determinism

Instrumentation cost separates the planes. A kernel-level tracepoint via LTTng imposes overhead on the order of tens to a few hundred nanoseconds per event and is explicitly engineered to be usable in production and real-time contexts [16]; `ros2_tracing` builds on it and reports overhead low enough for real-time ROS 2 systems, adding only a small percentage to message latency [17]. By contrast, an OpenTelemetry SDK span or measurement allocates on the heap, takes locks in the processor/exporter path, and batches for network export [1] — an architecture appropriate for millisecond requests but not for a microsecond jitter budget. The design conclusion is not “OTel is slow” but “OTel belongs on the non-real-time side of a lock-free boundary”: the hot path writes to a wait-free ring; a non-real-time thread drains it into the SDK.

7 Case Studies

7.1 Dapper: sampling is a feature of scale, not of correctness (cloud)

Google’s Dapper, the origin of modern distributed tracing and a direct ancestor of OpenTelemetry, sampled aggressively — retaining a small fraction of traces (on the order of $\frac{1}{1000}$ for high-volume services) — and still delivered its diagnostic value [5]. This works because in a data center the **population** of events is enormous and statistically homogeneous: a representative sample characterizes the whole. On a robot the diagnostically important events are **rare and non-representative** (the one overrun, the one dropped frame). Sampling optimizes for the common case; robotics reconstruction optimizes for the tail. This is the crispest statement of why the cloud sampling model must be **inverted** on the robot: record everything locally, **upload** a sample or a trigger.

7.2 ros2_tracing / LTTng: real-time tracing is possible, but it is not OTel (robotics)

The ros2_tracing framework demonstrates that low-overhead, real-time-safe tracing of a robotic middleware is achievable — by using static instrumentation points and a kernel-grade tracer (LTTng) rather than a general-purpose SDK [16], [17]. It is the existence proof for the recommended architecture: the on-robot causal/latency signal is captured by an RT-safe tracer, and OpenTelemetry, if used at all, consumes the **result** off-line. It also shows that the robotics community already has purpose-built tooling for the recording plane that OTel does not replace.

7.3 PREEMPT_RT and OSADL latency plots: the hiccup budget is microseconds (real-time)

Surveys and continuous measurement of PREEMPT_RT show worst-case scheduling latencies in the tens of microseconds on tuned hardware, versus millisecond-scale spikes on unconfigured mainline kernels [6], [7]. This quantifies the determinism plane’s budget and explains why instrumentation overhead and, critically, **page faults** (a major fault can cost milliseconds) must be engineered out of the control path (e.g. via `mlockall`, CPU isolation). An observability agent that can induce a page fault or a lock on the RT thread is not a diagnostic aid but a new fault source.

7.4 ROS 2 / DDS communications: dataflow latency is real and jitter-sensitive (robotics)

Independent evaluations of ROS 2 over DDS report end-to-end message latencies ranging from tens of microseconds to milliseconds depending on QoS, transport, and payload, with jitter that matters for control [18], [19]. Two implications follow. First, the causality a robot engineer needs is **dataflow provenance** across these pub/sub hops, not an RPC call tree — so trace identity must ride in the message envelope, and DDS’s own QoS (**deadline**, **liveliness**) is a native liveness signal to harvest [13]. Second, systems like Autoware run this stack on embedded compute [20], where the heavy OTLP/gRPC exporter’s footprint is a real constraint, reinforcing the API-in-core / SDK-in-sidecar split.

7.5 Pressure Stall Information: the cloud-native community built the right hiccup detector (transferable)

Linux PSI, contributed from hyperscale operations, exposes per-resource stall time for CPU, memory, and I/O — a direct, low-overhead measure of “is something being starved” [21]. It is a canonical example of a cloud-native artifact that transfers **directly** to robotics: the same signal that detects a noisy-neighbour container also detects a planner starved by a log flush. Together with eBPF-based introspection [22], PSI is exactly the kind of host-telemetry the observability plane should collect and export via OTLP.

8 Gap Analysis: What Robotics Must Add

OpenTelemetry supplies the observability plane; the following are the pieces a robotics stack must build or borrow elsewhere for the whole to be valuable.

1. **Real-time-safe front-end.** A wait-free capture ring on the hot path, drained off-RT into the SDK. OTel has no hard-real-time story; this is legitimately the robot’s to own.
2. **A hardware time model.** Monotonic clocks for intervals, **hardware capture timestamps** (sample time $\neq$ ingestion time — the distinction that governs sensor fusion), and PTP/gPTP cross-device synchronization. OTel attributes can **carry** these but do not define them.
3. **Lossless flight recorder.** An always-on bounded ring snapshotted pre/post trigger on any anomaly, fault, or e-stop — the inverse of sampling, stored in MCAP [14], not the metrics pipeline.
4. **A high-rate signal store distinct from metrics** (§6.2).

5. **Dataflow-native context propagation.** Correlation identity attached to messages/frames across DDS/ROS/shared memory (§7.4), not HTTP headers.
6. **Robotics semantic conventions.** `robot.joint.*`, `sensor.*`, `battery.*`, `motor.*`, `safety.state`, `frame_id` — with prior art in ROS 2 diagnostics [23] and a candidate for upstreaming to OTel’s convention process.
7. **Health aggregation with hysteresis**, fusing per-device health and host resources into a roll-up tree (semantics from `diagnostic_aggregator` [23]), exported as OTel metrics.
8. **Boot/lifecycle and offline-first capture**, retained locally and shipped opportunistically via the Collector’s persistent queue.
9. **Strict observability/safety separation** (§3): the safety monitor remains independent of OTel.

9 Related Work: Existing Building Blocks

Before designing anything, we surveyed whether the real-time capture layer that §8 identifies as differentiating already exists as adoptable software. It does, and at high maturity (Table 4). The wait-free ring with flight-recorder snapshot — which one might assume is bespoke — is precisely LTTng’s documented design [16] and ROS 2’s `rosbag2` snapshot mode [24]; the deferred-format hot-path logging that the “capture raw, decode off-line” pattern (§6.3) describes is shipped by NanoLog [25] and Quill [26] at nanosecond scale.

Layer	Solution	ROS-coupled?	Verdict
Wait-free ring + flight recorder	LTTng UST, snapshot mode [16]	No	Adopt
RT logging (deferred format)	NanoLog [25], Quill [26]	No	Adopt
RT tracing for robotics	<code>ros2_tracing</code> [17], <code>ros2probe</code> [27]	Yes	Adopt if ROS
Recording format	MCAP [14]	No	Adopt
Snapshot-to-disk on trigger	<code>rosbag2</code> snapshot [24]	Yes	Pattern to mirror
Zero-copy RT transport	Eclipse iceoryx [28]	No	Adopt if needed
Fleet export + backends	OpenTelemetry [1], Foxglove [29], Sift [30]	No	Adopt
OTel↔ROS bridge	<code>ros-opentelemetry</code> [31]	Yes	Reference

Table 4: Adoptable building blocks per layer. The real-time-capture primitives are mature and framework-agnostic; most turnkey robotics observability is coupled to ROS 2.

Two conclusions follow. First, the capture engine must be **adopted, not reimplemented**: reproducing lockless multi-core buffering or nanosecond deferred logging would re-derive published, battle-tested work. Second, a **framework** split emerges: the turnkey robotics tools (`ros2_tracing`, `rosbag2`, `ros-opentelemetry`) assume ROS 2, whereas the substrate primitives (LTTng, NanoLog/Quill, MCAP, OpenTelemetry, iceoryx) are framework-agnostic and adoptable by a non-ROS C++ family such as xMotion. The design task is therefore not to build an engine but to **bind** these engines coherently to the xMotion HAL without bloat — the subject of §10.

10 A Coherent, Lightweight Instrumentation Strategy

The objective is a single, thin instrumentation **surface** usable identically from control, planning, and decision code, with the §9 engines plugged in behind it. The organizing principle: **coherence**

comes from one spine; lightness comes from pluggable, compile-time-selected, off-by-default sinks. We design the spine and adopt the engines.

10.1 One surface, many sinks

Call sites bind to a minimal first-party surface — roughly four verbs: **event** (discrete structured record), **metric** (counter/gauge/histogram), **scope** (causal timing span with a correlation id), and **signal** (high-rate typed sample). Domain **health** is not a fifth primitive but a convention expressed over **metric** and **event**. The surface must be **smaller than any single backend’s API** — minimalism is the anti-bloat constraint — yet **shaped to the OpenTelemetry data model**, so each backend adapter is a near-zero-cost mapping rather than a fat translation layer. Binding call sites to the surface (not to LTTng’s or OTel’s API directly) is the same decoupling the family already applies to its logging vendor.

10.2 What to own versus what to adopt

Own and design (the spine)	Adopt (the engines)
The 4-verb instrumentation surface	Wait-free ring / flight recorder: LTTng [16]
Monotonic clock, run/session id, resource identity	Deferred-format RT logging: NanoLog/Quill [25], [26]
Correlation-id-in-message convention (not HTTP)	Recording format: MCAP [14]
RT capture boundary contract (wait-free produce, drop policy)	Export + backends: OpenTelemetry/OTLP [4]
Domain vocabulary: health / freshness / deadline-miss	Time synchronization: PTP/gPTP
Routing policy: primitive $\rightarrow$ plane/sink	Correlation-id encoding: W3C Trace Context [3]

Table 5: The division of labour. The owned “spine” is small and specific to the xMotion HAL; the adopted engines are the mature commodity substrate.

10.3 Three rules that keep it lightweight

1. **Thin, adapted, not bound.** Call sites use the four verbs; backends are adapted behind them. The surface stays OTel-shaped so adapters are trivial, but no call site ever includes a backend header.
2. **Compile-time selection and no-op-when-off.** A build links only the sinks it enables; disabled instrumentation compiles to nothing (NanoLog-style compile-time extraction). “Integrate everywhere” must not mean “everywhere pays.”
3. **Header-light core, heavy sinks isolated.** The surface is a light header plus a small runtime in the foundation with no LTTng/OTel/MCAP dependency; sinks are separate optional targets. This is the OpenTelemetry API/SDK split applied to the family’s own surface.

10.4 Two data classes, one surface

The same four verbs carry two data classes with different fates: **aggregatable diagnostics** (**event/metric/health**) routed to the observability plane (OpenTelemetry), and **raw high-rate signals** (**signal**) routed to the recording plane (MCAP/LTTng). The routing policy — not the call site — separates them, so a 1 kHz raw stream never enters the metrics pipeline (§6.2). A discipline test follows directly: if adding a backend requires touching call sites, the surface has leaked; if instrumenting a new module pulls a heavy dependency, the core has leaked. Either is a design defect, not a feature.

A strategic corollary: because the surface hides the sinks, the layer can be built before the remaining engine choice (LTTng versus a lock-free ring on lean targets) is made — the call sites do not change when it is resolved later. The design is deliberately decision-independent, which de-risks it.

10.5 ROS-free core, ROS-bridgeable edge

The library is deliberately **ROS-free**: it depends on none of `rclcpp/rmw/DDS`, `rosbag2`, `ros2_tracing`, or `ament`, so it links identically into non-ROS control code and into ROS 2 nodes. This honours the family’s downward-only dependency rule — ROS lives at the application layer as a **consumer**, not a dependency of the foundation. ROS-free is not ROS-hostile: three seams keep the core ROS-free while remaining fully interoperable with a ROS 2 application.

Time the core stamps with a plain monotonic clock and hardware capture time; mapping to ROS time (including simulated `/clock` time) is done by the application at the boundary, not baked into the library — no `rclcpp::Clock` dependency.

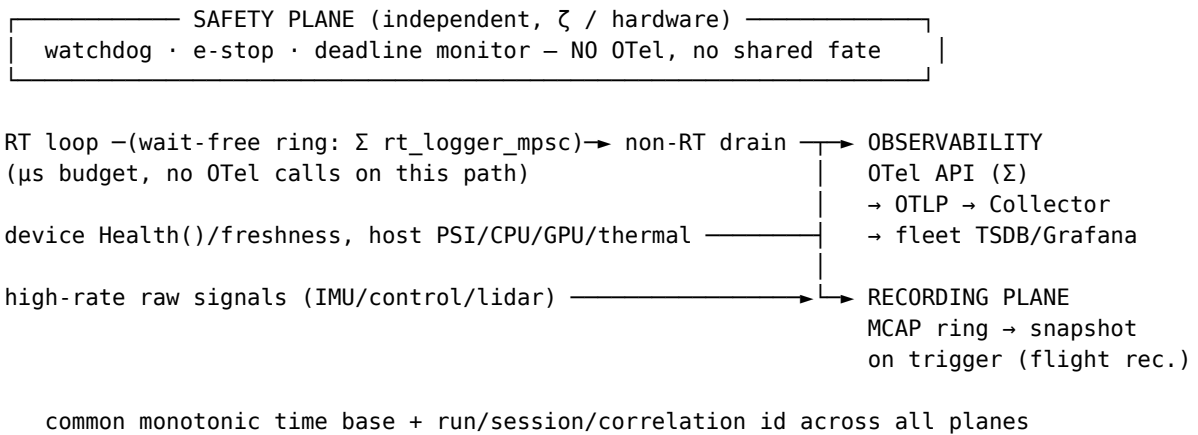
Correlation id the core owns the id **type/format** (W3C-shaped) and the get/set-current-context helpers, but not the **carrier**. A non-ROS component places the id in its DDS/shared-memory envelope; a ROS 2 node places it in a message-header field. The library never includes a ROS message type.

Recording MCAP is a serialization-agnostic **format**, not a ROS component (`rosbag2` is the ROS wrapper around it). The core writes MCAP directly with a neutral encoding, yielding artifacts the ROS ecosystem and Foxglove can open — a ROS-free producer with a ROS-compatible artifact.

Any genuinely ROS-specific glue — injecting ROS time, reading the id from a message header, exporting to a ROS diagnostics topic or a `rosbag2/ros2_tracing` session — lives in an **optional application-side bridge** that depends on ROS, never in the library. Because LTTng is the tracing substrate beneath both this library and `ros2_tracing` [17], a ROS application can even correlate the two in a single LTTng session without the library knowing ROS exists. This decision fixes the engine set: the library adopts only the framework-agnostic engines (LTTng, NanoLog/Quill, MCAP, OpenTelemetry); the ROS-coupled tools of Table 4 become optional application-side bridges, not library dependencies.

11 Recommended Architecture for the xMotion Family

The three-plane model maps onto the xMotion components (Σ foundation, μ drivers, ∇ algorithms, γ visualization, ζ firmware) as shown in Listing 1.



Listing 1: Three-plane telemetry architecture mapped onto the xMotion family. One monotonic time base and run/correlation id thread all planes.

Σ (**foundation**) owns the **spine** of §10 — the four-verb instrumentation surface, the monotonic time base, the run/correlation-id convention, the domain health vocabulary, and the RT capture

boundary contract — as a light header plus small runtime with **no** heavy telemetry dependency in core.

New telemetry component (optional, off by default) the adopted **engines** as sinks behind the surface — LTTng/NanoLog for RT capture, MCAP for recording, the OpenTelemetry SDK + Collector for export — plus the host/GPU/thermal collectors. Compile-time-selected; depends on Σ .

∇ (algorithms) the reaction/policy layer — fuse health, enforce budgets, arm the recorder, degrade or failsafe. The only layer permitted to act on telemetry.

γ (quickviz) live on-robot visualization; Foxglove/Grafana for offline and fleet analysis.

ζ (firmware) + application/umbrella the independent safety backstop; the composition root that selects exporters, budgets, and retention; and the home of any optional ROS bridge (§10.5) — ROS-specific glue lives here, never in the ROS-free library.

The guiding invariant restates the safety argument in dependency terms: telemetry primitives point **down** into Σ , collectors are **optional peers**, and only ∇ may **react**. A collector that could trip a failsafe directly would couple observation to actuation — exactly what independence forbids.

12 Threats to Validity

Our quantitative claims are order-of-magnitude arguments built from published measurements on **representative** platforms, not a controlled benchmark of the xMotion stack; absolute overhead and latency figures are implementation- and hardware-dependent (kernel configuration, DDS vendor, SoC). The specific sampling ratio attributed to Dapper is illustrative of a policy, not a universal constant. The three-plane model is a design heuristic; some concerns (e.g. anomaly-triggered trace export) legitimately straddle the recording and observability planes. These caveats do not affect the central, direction-level conclusions, which rest on qualitative architectural properties (loss tolerance, determinism, causality shape) rather than precise numbers.

13 Conclusion

OpenTelemetry is neither a silver bullet nor an ill fit for robotics — it is a precise fit for one of three planes. For the **observability plane**, adopt it directly: OTLP, the Collector, the metrics/resource model, host and hardware semantic conventions, and the backend ecosystem are a mature capability robotics should not rebuild. For **dataflow tracing** and **health metrics**, borrow the model and adapt the mechanism. For the **real-time hot path** and the **safety monitor**, keep OpenTelemetry out entirely; those belong to a wait-free capture lane and an independent safety channel respectively, and the high-rate recording plane belongs to MCAP. Quantitatively, OpenTelemetry is a strong adopt for roughly 40% of the problem, a borrow-and-adapt for 30%, and irrelevant-to-harmful for the 30% that is the control loop and the safety case. The cloud-native community’s durable gifts to robotics — OTLP and the Collector for transport and buffering, eBPF and PSI for low-overhead introspection, and the discipline of semantic conventions — are real; realizing them requires filling the robotics-specific gaps of real-time-safe capture, a hardware time model, lossless anomaly recording, and dataflow-native causality, unified by a single time base and correlation identity across all three planes.

A Host telemetry data sources (Linux)

Signal	Source
Per-core CPU	/proc/stat, /proc/schedstat
Resource stall (hiccups)	/proc/pressure/{cpu,memory,io} (PSI) [21]
Memory	/proc/meminfo, cgroup memory.stat
Disk I/O + latency	/proc/diskstats
Network	/proc/net/dev, socket drop counters
Thermal	/sys/class/thermal, hwmon
GPU	NVML (desktop), tegrastats (Jetson), ROCm-SMI (AMD)
Scheduling latency / jitter	cyclictest, direct loop-wakeup measurement [7]
Kernel/syscall/sched introspection	eBPF (bcc, bpftrace) [22]

Table 6: Low-overhead kernel interfaces for observability-plane collection.

B Proposed `robot.*` semantic-convention seeds

Candidate attribute names, aligned in spirit with OpenTelemetry conventions and seeded from ROS 2 diagnostics [23]: `robot.id`, `robot.frame_id`, `robot.joint.name`, `robot.motor.current_a`, `robot.motor.temperature_c`, `robot.sensor.staleness_ms`, `robot.battery.state_of_charge`, `robot.safety.state`, `robot.mission.phase`.

References

- [1] OpenTelemetry Authors (CNCF), “OpenTelemetry Specification.” 2024.
- [2] OpenTelemetry Authors (CNCF), “Host Metrics Receiver and System/Hardware Semantic Conventions.” 2024.
- [3] World Wide Web Consortium (W3C), “Trace Context.” 2021.
- [4] OpenTelemetry Authors (CNCF), “OpenTelemetry Protocol (OTLP) Specification.” 2024.
- [5] B. H. Sigelman *et al.*, “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” technical report dapper-2010-1, 2010.
- [6] F. Reghenzani, G. Massari, and W. Fornaciari, “The Real-Time Linux Kernel: A Survey on PREEMPT_RT,” *ACM Computing Surveys*, vol. 52, no. 1, pp. 1–36, 2019.
- [7] OSADL, “QA Farm on Real-time of Mainline Linux — Latency plots.” 2024.
- [8] International Organization for Standardization, “ISO 26262: Road vehicles — Functional safety.” 2018.
- [9] International Electrotechnical Commission, “IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems.” 2010.
- [10] RTCA, “DO-178C: Software Considerations in Airborne Systems and Equipment Certification.” 2011.
- [11] ARINC / Aeronautical Radio, Inc., “ARINC Specification 653: Avionics Application Software Standard Interface.” 2019.
- [12] AUTOSAR, “Specification of Watchdog Manager, AUTOSAR Classic Platform.” 2023.

- [13] Object Management Group, “Data Distribution Service (DDS), Version 1.4,” technical report formal/2015–4–10, 2015.
- [14] Foxglove, “MCAP: A serialization-agnostic container file format for pub/sub messages.” 2022.
- [15] Prometheus Authors, “Metric and label naming; instrumentation best practices.” 2024.
- [16] M. Desnoyers and M. R. Dagenais, “The LTTng tracer: A low impact performance and behavior monitor for GNU/Linux,” in *Proceedings of the Ottawa Linux Symposium (OLS)*, 2006.
- [17] C. Bédard, I. Lütkebohle, and M. Dagenais, “ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6511–6518, 2022.
- [18] Y. Maruyama, S. Kato, and T. Azumi, “Exploring the performance of ROS2,” in *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*, 2016.
- [19] C. S. V. Gutiérrez, L. U. S. Juan, I. Z. Ugarte, and V. M. Vilches, “Towards a distributed and real-time framework for robots: Evaluation of ROS 2.0 communications for real-time robotic applications,” *arXiv preprint arXiv:1809.02595*, 2018.
- [20] S. Kato *et al.*, “Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems,” in *ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS)*, 2018.
- [21] The Linux Kernel Documentation, “PSI — Pressure Stall Information.” 2018.
- [22] B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley, 2019.
- [23] Open Robotics, “ROS 2 diagnostics: diagnostic_updater and diagnostic_aggregator.” 2024.
- [24] Open Robotics, “rosv2 snapshot mode design.” 2024.
- [25] S. Yang, S. J. Park, and J. Ousterhout, “NanoLog: A nanosecond scale logging system for C++.” 2018.
- [26] O. Loukas, “Quill: Asynchronous Low-Latency C++ Logging Library.” 2024.
- [27] Anonymous, “ros2probe: Non-intrusive, Kernel-selective Observability for ROS 2 Middleware.” 2026.
- [28] Eclipse Foundation, “Eclipse iceoryx: true zero-copy inter-process communication.” 2024.
- [29] Foxglove, “Foxglove: multimodal data platform for robotics.” 2024.
- [30] Sift, “Sift: observability and telemetry monitoring for robotics.” 2024.
- [31] S. Zobov, “ROS2 × OpenTelemetry: End-to-End Telemetry for Robotics”. 2025.